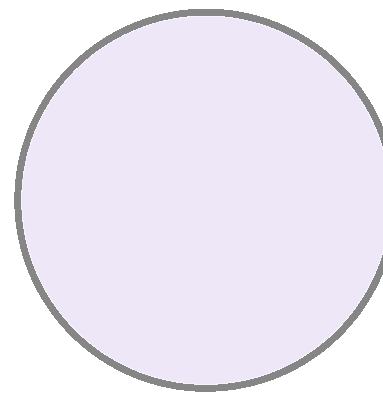




TRACKLY

Technical Documentation

Architecture, Development, Features, and
Operations



VERSION

1.0

AUTHOR

Fahmy Akhmad
Firdaus

YEAR

2026

Table of Contents

Document Information

Preface

How to Read This Document

Part I — Product and Architecture

1 Project Overview

- 1.1 Product Goals
- 1.2 Implemented Scope
- 1.3 Current Status

2 System Architecture

3 Technology Stack

4 Repository Structure

Part II — Application Design

5 Frontend Architecture

- 5.1 Routing and Layout
- 5.2 Data and State

6 Backend Architecture

7 Authentication and Authorization

8 Database Design

9 API Design

Part III — Features

10 Today Dashboard

11 Habits

12 Goals

13 Categories and Preferences

14 Analytics

15 Reminders and Web Push

Part IV — Engineering

16 Local Development

17 Environment Configuration

18 Testing

19 Debugging

20 Docker and Database Migrations

Part V — Deployment and Operations

21 Production Deployment

22 Reverse Proxy and Networking

23 Monitoring and Logging

24 Backup and Recovery

25 Security and Operational Readiness

Part VI — Reference

26 API Reference

27 Database Reference

28 Troubleshooting

29 Frequently Asked Questions

- 29.1 Is analytics persisted?
- 29.2 Does the backend start the scheduler?
- 29.3 Can a client choose a user ID?
- 29.4 Can `db:push` deploy production schema?
- 29.5 Is a Compose volume a backup?
- 29.6 Does Trackly use Firebase?

30 Glossary

31 Decision Log

Appendices

Appendix A — Useful Commands

Appendix B — Known Limitations

Appendix C — Future Improvements

Appendix D — Documentation Maintenance

Document Information

Field	Value
Title	Trackly Technical Documentation
Subtitle	Architecture, Development, Features, and Operations
Author	Fahmy Akhmad Firdaus
Version	1.0
Year	2026
Language	English
Primary format	A4 portrait
Source	Curated from the Trackly repository and canonical Markdown documentation

This book describes the implementation present in the Trackly repository in 2026. It does not reconstruct undocumented project history, promise uncommitted features, or replace the canonical engineering documents under `docs/`.

Preface

Trackly is a personal productivity platform built around habits, goals, reminders, and derived analytics. Its implementation pairs a server-rendered Next.js application with a versioned Fastify API and PostgreSQL. The system is designed around authenticated ownership, logical calendar dates, explicit validation, and derived rather than persisted metrics.

This publication explains how those choices work together. Product chapters show the user-facing behavior; design chapters describe the boundaries behind it; engineering and operations chapters explain how to run, validate, and deploy it safely.

Tasks and the standalone Insights route are intentionally identified as unimplemented placeholders. Their navigation presence is not evidence of a completed product module.

How to Read This Document

- Read Parts I and II for architectural orientation.
- Use Part III to understand product workflows and business rules.
- Use Part IV while developing or reviewing changes.

- Use Part V before operating Trackly outside local development.
- Use Part VI for contracts, schema lookup, troubleshooting, and terminology.

The API and database chapters are compact publication references. The canonical [API reference](#) and [database schema](#) remain the exhaustive sources.

Note

Markdown is the source of truth. Mermaid figures in this manuscript are rendering inputs and must be converted to SVG before the final PDF or DOCX is produced.

Part I — Product and Architecture

1 Project Overview

Trackly combines habit tracking, task-oriented daily context, goal tracking, reminders, preferences, notifications, and analytics in one authenticated application. The implemented center of gravity is habit consistency: schedules produce logical occurrences; check-ins record absolute progress; goals and analytics derive meaning from those source records.

1.1 Product Goals

- Give users a calm daily view of scheduled work.
- Make recurring progress explicit without retaining meaningless zero rows.
- Keep data isolated by authenticated ownership.
- Respect user-local calendar boundaries across Today, schedules, streaks, reminders, and analytics.
- Provide extension points without placing business logic in transport or UI infrastructure.

1.2 Implemented Scope

Authentication, Today, categories, habits, goals, analytics, preferences, reminders, durable notification delivery, and browser Web Push are implemented. Tasks have a database/query foundation and placeholder page rather than a completed workflow. Standalone Insights is also a placeholder; analytics insights are implemented inside the Analytics dashboard.

1.3 Current Status

The repository includes production build targets, database migrations, structured logs, health/readiness checks, security controls, and extensive unit/integration coverage. It does not include production orchestration, TLS termination, automated backups, external monitoring, or deployment automation.

2 System Architecture

Trackly separates browser concerns, server rendering, API transport, domain services, repositories, and persistent data. Authentication is delegated to Better Auth. Reminder processing runs as a separate backend runtime, while Web Push remains behind a provider-neutral dispatcher.

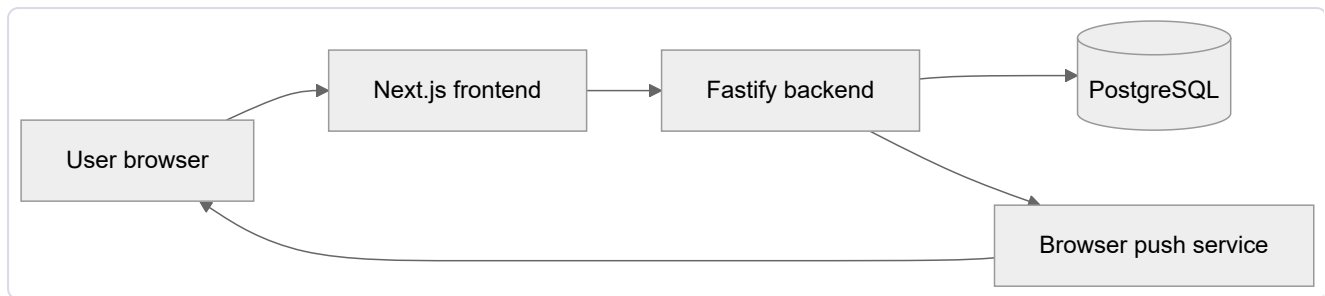


Figure 2.1 — Trackly System Context

The frontend does not query PostgreSQL. The API does not place SQL in routes or controllers. User-specific reads avoid shared static caches, and ownership filters are applied in repositories.

3 Technology Stack

Area	Technology	Role
Frontend	Next.js 16, React 19, TypeScript	App Router, server rendering, focused client interaction
Styling	Tailwind CSS 4, shadcn/ui conventions, Lucide	Responsive design system and icons
Forms	React Hook Form, Zod	Typed forms and validation
HTTP	Axios plus server request utility	Central authenticated frontend API access
Charts	Recharts	Accessible analytics trends
Backend	Fastify 5, TypeScript	Plugin-oriented HTTP API
Validation	Zod	Params, query, body, environment, and public configuration
Database	PostgreSQL 17	Relational ownership and logical-date storage
ORM	Drizzle ORM	Typed schema, queries, relations, and migrations
Authentication	Better Auth	Users, accounts, sessions, verification, cookies
Testing	Vitest, Testing Library, Fastify injection, Playwright tooling	Unit, integration, UI, and publication capture
Runtime	Docker, Docker Compose, pnpm workspaces	Repeatable development and production build stages

The stack favors explicit TypeScript contracts and libraries that work in both local development and containerized deployment.

4 Repository Structure

```
Trackely/
├─ backend/           Fastify API, scheduler, database, and tests
├─ frontend/          Next.js application and UI tests
├─ docs/              Canonical and publication documentation
│   └─ 00-overview/
│   └─ 01-design/
│   └─ 02-features/
│   └─ 03-development/
│   └─ 04-deployment/
│   └─ 05-reference/
│   └─ assets/
│   └─ publication/
├─ scripts/docs-screenshots/
├─ docker-compose.yml
├─ package.json
├─ pnpm-workspace.yaml
└─ AGENTS.md
```

The workspace contains two application packages rather than an `apps/` or shared `packages/` monorepo hierarchy. Database schema and migrations live inside the backend. Repository-wide commands orchestrate both packages.

Part II — Application Design

5 Frontend Architecture

The frontend uses the Next.js App Router and defaults to React Server Components. Protected route groups obtain the Better Auth session server-side, fetch user-specific data with `no-store`, and render stable initial UI. Client Components are limited to forms, mutations, theme control, check-ins, reminder management, and browser Web Push.

5.1 Routing and Layout

Public authentication pages use a dedicated layout. Authenticated routes share an application shell with desktop sidebar, top bar, responsive content area, and fixed mobile navigation. Route-level loading, error, and not-found files provide localized recovery.

5.2 Data and State

- Shareable list and analytics selections live in URL parameters.
- React Hook Form owns temporary form state.
- Focused component state manages pending and feedback behavior.
- No global client data cache or React Query layer is present.
- Server requests use a shared parser, timeout, cookie forwarding, and safe error mapping.

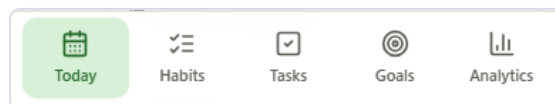


Figure 5.1 — Mobile Primary Navigation

6 Backend Architecture

Fastify is assembled in `app.ts` without opening a port. `server.ts` owns startup and graceful shutdown. Plugins register authentication, database access, request context, security, CORS, rate limiting, Swagger, and centralized errors.

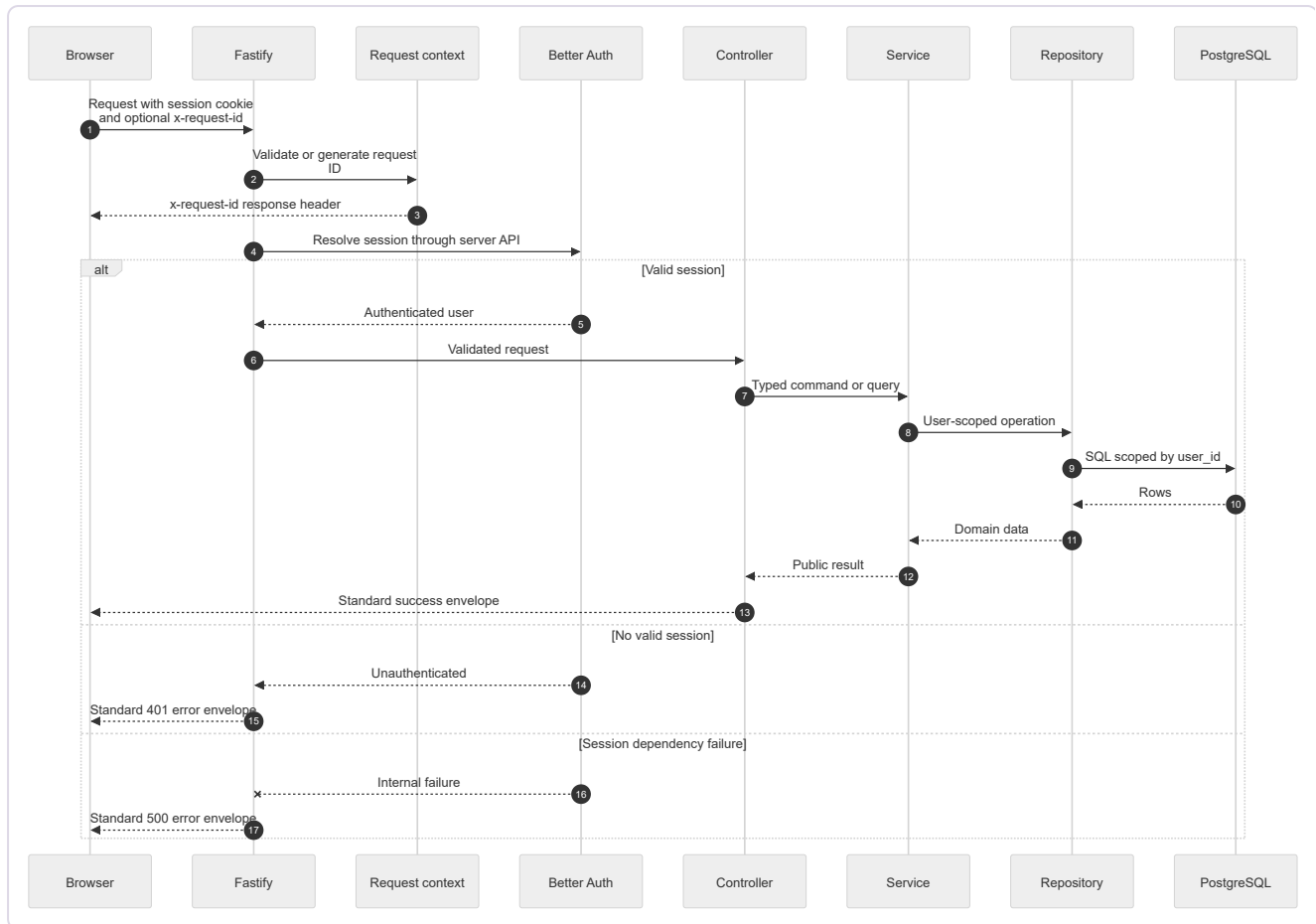


Figure 6.1 — Authenticated Request Lifecycle

Controllers translate HTTP input and output. Services enforce business, schedule, and timezone rules. Repositories own database access. This lightweight CQRS separation is most visible in the Habit read and command paths.

7 Authentication and Authorization

Better Auth owns password hashing, accounts, sessions, verification records, cookies, and `/api/auth/*`. Trackly resolves sessions through Better Auth's server API rather than decoding cookies. Protected pages redirect to login; protected API routes return the standard 401 error envelope.

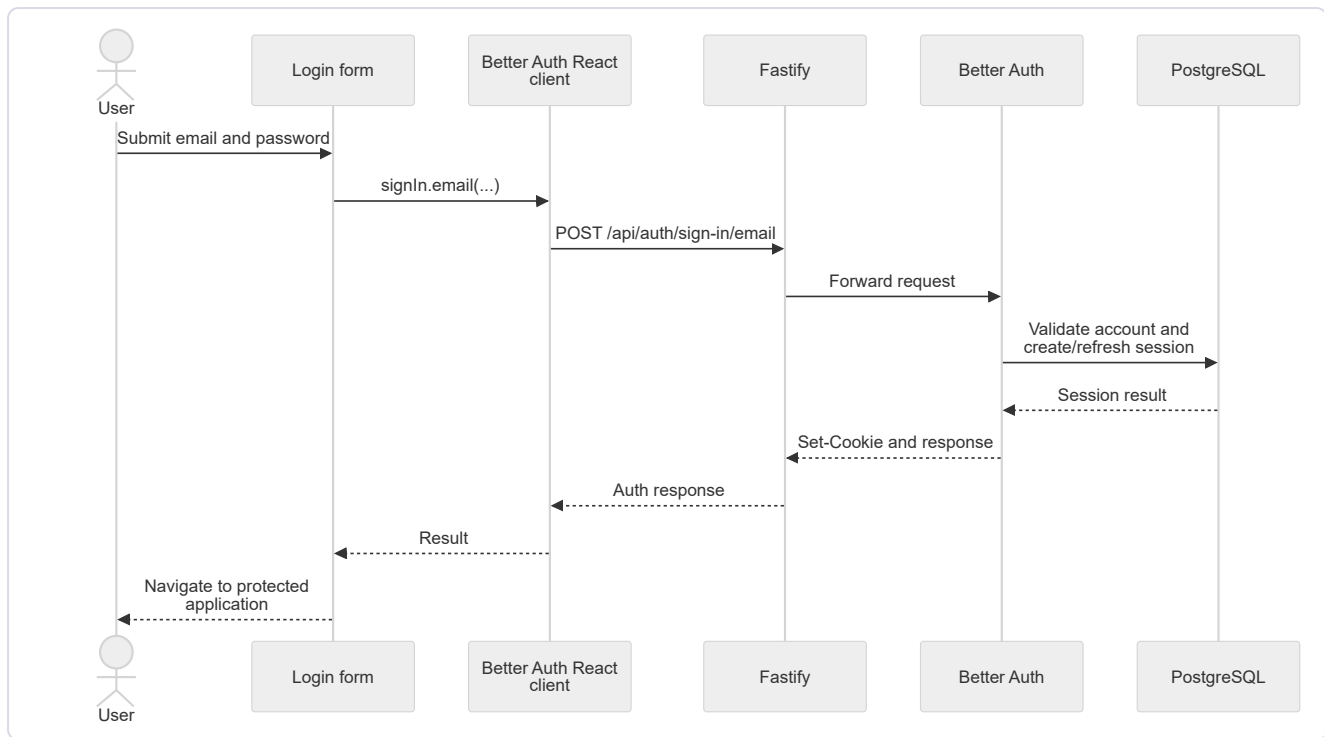
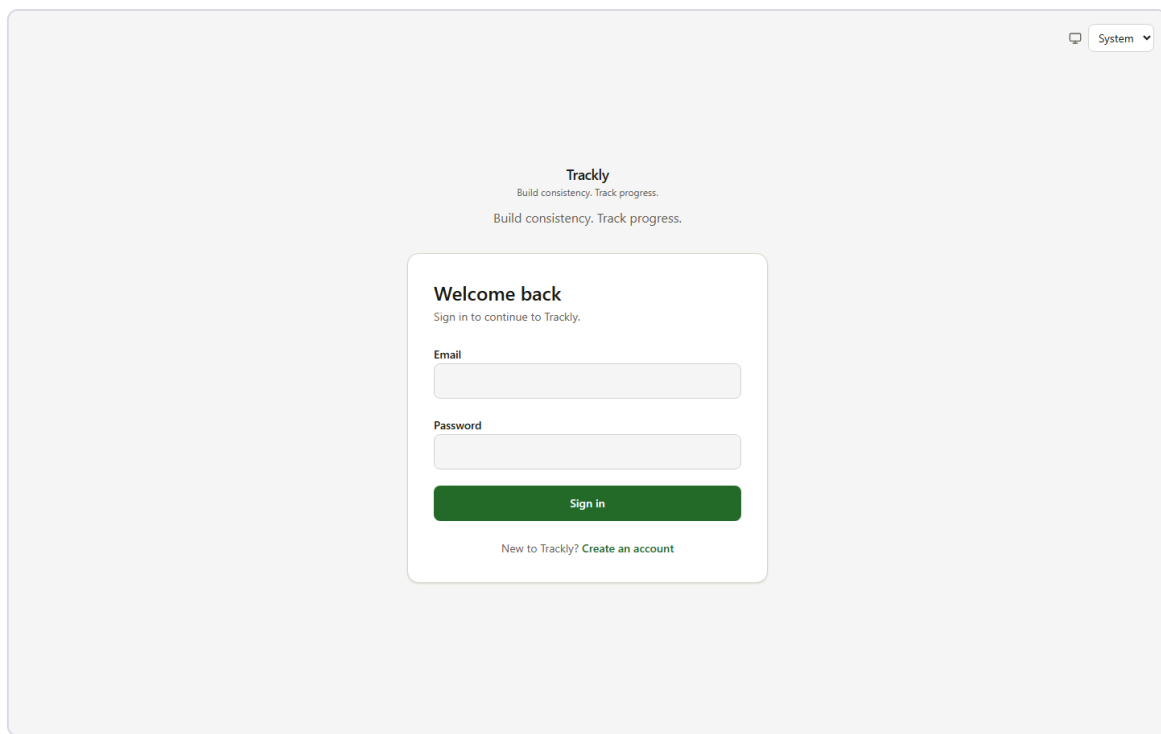


Figure 7.3 — Email and Password Authentication Sequence

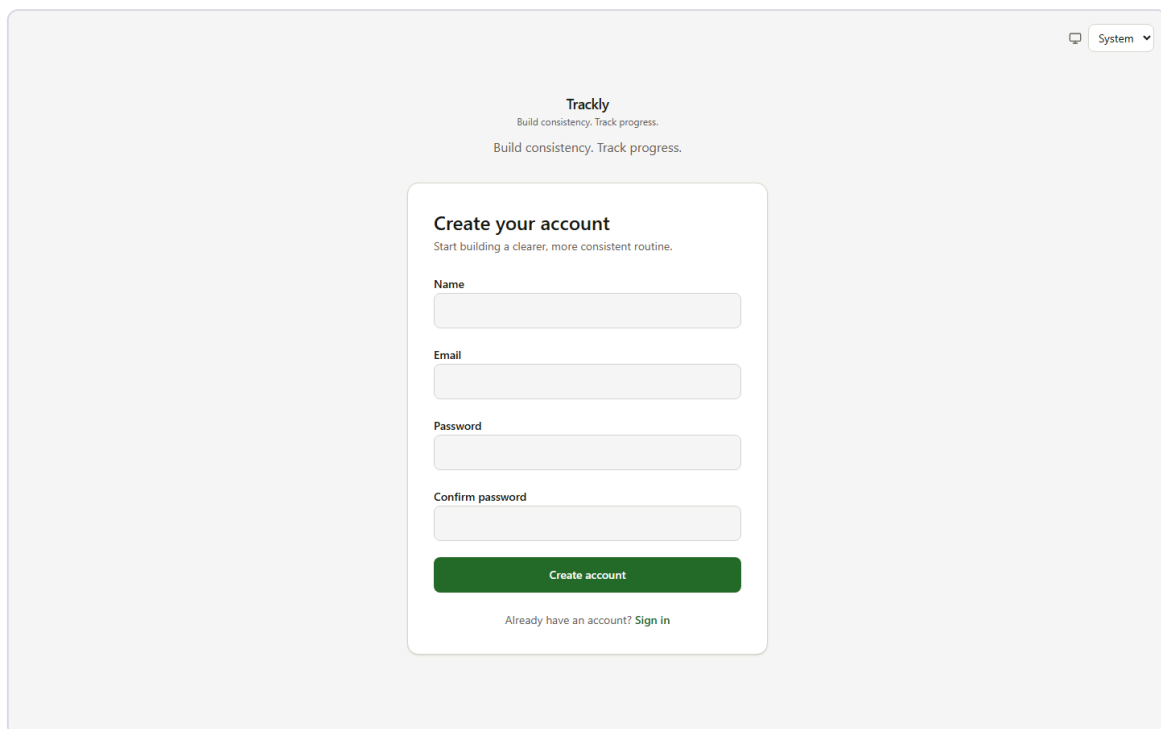
Authorization is ownership-based. The authenticated user ID is passed into services and repositories; public request bodies never choose an owner. Deleted, missing, and foreign habit records intentionally share not-found behavior.



The image shows a web application interface for Trackly. At the top right, there is a "System" dropdown menu. The main heading is "Trackly" with the tagline "Build consistency. Track progress." repeated twice. Below this is a "Welcome back" section with the instruction "Sign in to continue to Trackly." The form contains two input fields: "Email" and "Password". Below these is a green "Sign in" button. At the bottom of the form, it says "New to Trackly? [Create an account](#)".

Figure 7.1 — Trackly Sign-In Experience

Registration uses the same authentication-owned contract and creates a default preference row for the new user.



The image shows a web application interface for Trackly. At the top right, there is a "System" dropdown menu. The main heading is "Trackly" with the tagline "Build consistency. Track progress." repeated twice. Below this is a "Create your account" section with the instruction "Start building a clearer, more consistent routine." The form contains four input fields: "Name", "Email", "Password", and "Confirm password". Below these is a green "Create account" button. At the bottom of the form, it says "Already have an account? [Sign in](#)".

Figure 7.2 — Account Registration

8 Database Design

PostgreSQL stores authentication records, user preferences, categories, habits, schedules, check-ins, tasks, goals, goal steps, reminders, notification deliveries, and browser push subscriptions. Drizzle defines tables, relations, constraints, indexes, and migration snapshots.

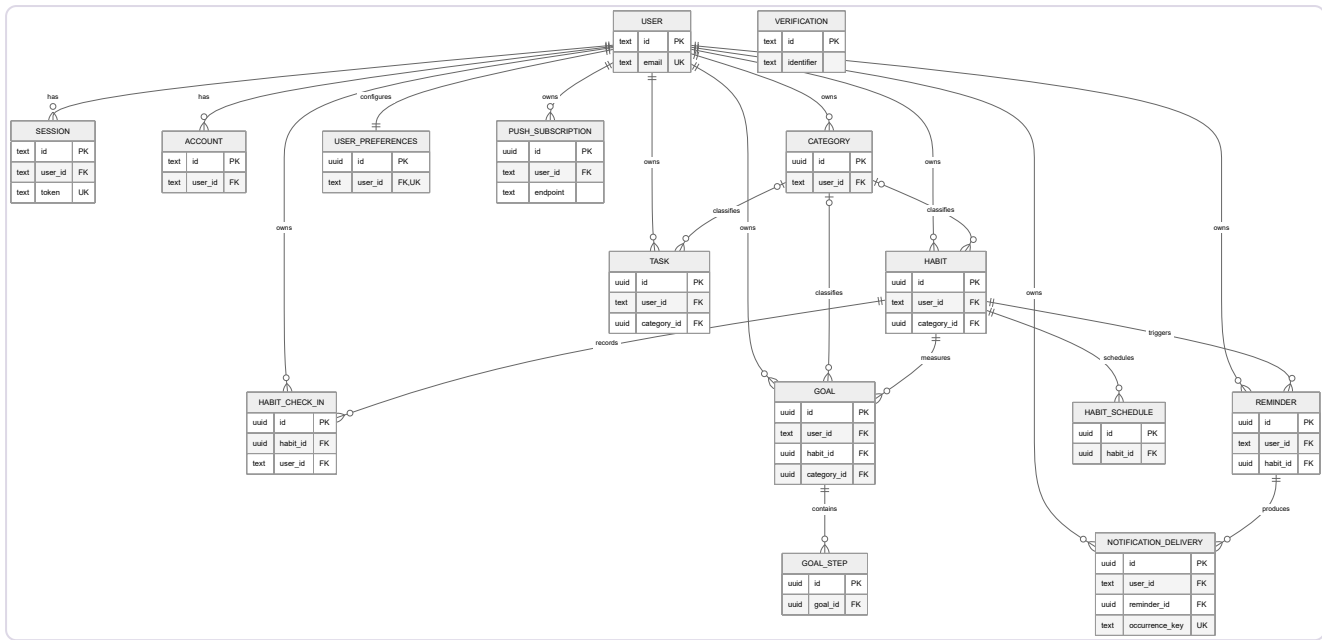


Figure 8.1 — Trackly Entity Relationships

Tables and columns use snake_case. UUIDs identify Trackly domain entities; Better Auth retains its own text identifiers. Logical days use PostgreSQL `date`, while instants use timezone-aware timestamps. Soft-deletable resources carry nullable `deleted_at`.

Migrations are reviewed and forward-only. `db:push` is limited to local development. Transactions protect dependent mutation steps, occurrence claims, and other atomic workflows.

9 API Design

Trackly application endpoints live under `/api/v1`. Better Auth owns `/api/auth/*`; `/health`, `/ready`, and conditionally exposed `/docs` remain unversioned infrastructure routes.

Success responses use:

```
{
  "success": true,
  "data": {},
  "meta": {}
}
```

Errors use stable codes and JSON-safe optional details:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Request validation failed.",
    "details": []
  }
}
```

Zod validates params, query strings, bodies, and configuration. The centralized handler maps Zod, Fastify validation, malformed JSON, application errors, missing routes, and unexpected failures without exposing stack traces or raw database errors.

Part III — Features

10 Today Dashboard

Today is the user's local-calendar command center. It combines scheduled habits, task group projections, active goals, and a daily summary. A URL date parameter supports explicit date navigation without losing refresh or browser-history state.

Habit progress uses the source check-in count for the selected date. Tasks are represented only through their existing query foundation; no mutation UI is present. Goal cards show step summaries, while detailed goal progress is derived elsewhere from habit check-ins.

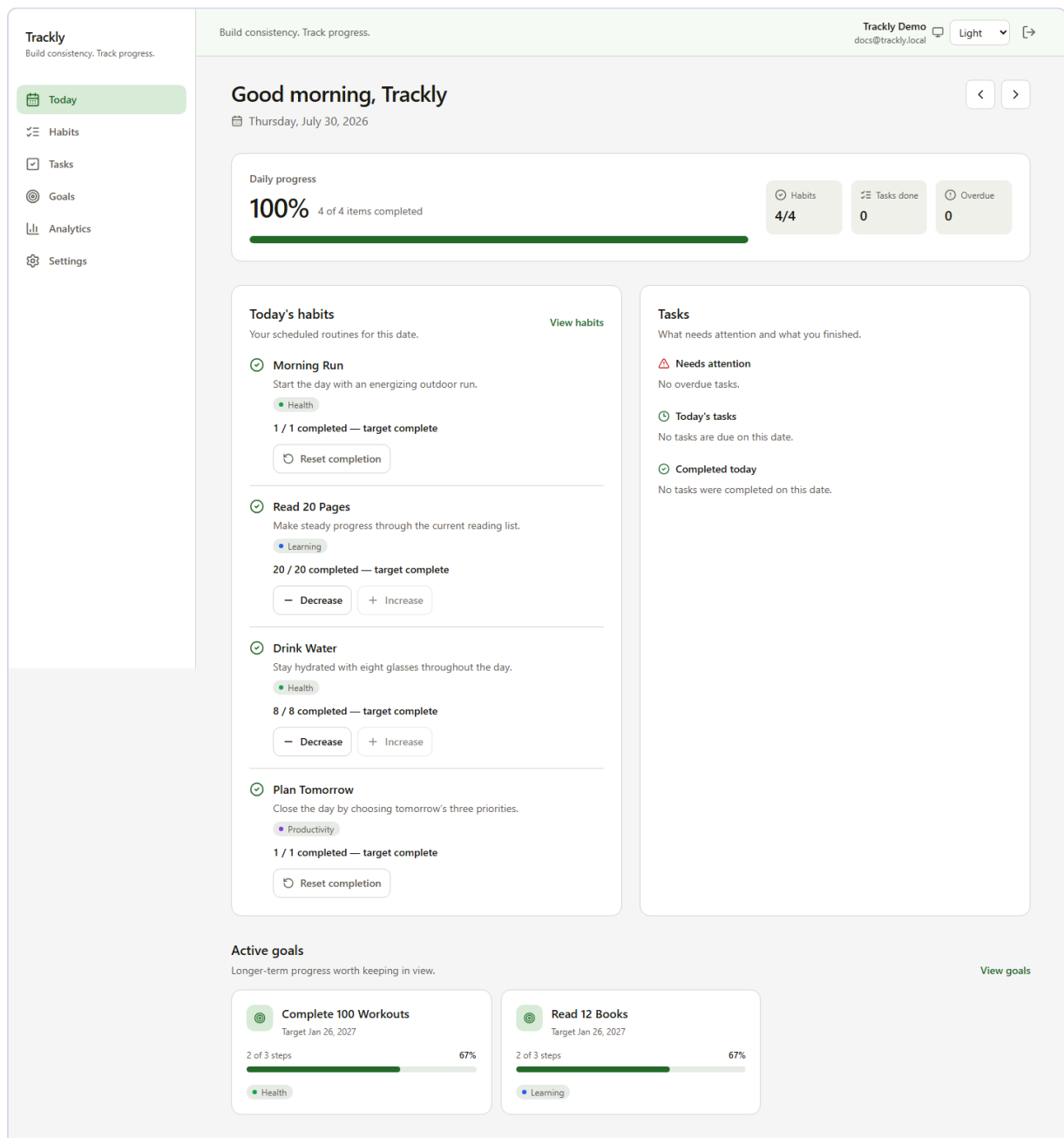


Figure 10.1 — Today Dashboard with Daily Progress

On narrow screens, cards stack and mobile navigation remains fixed without horizontal overflow.

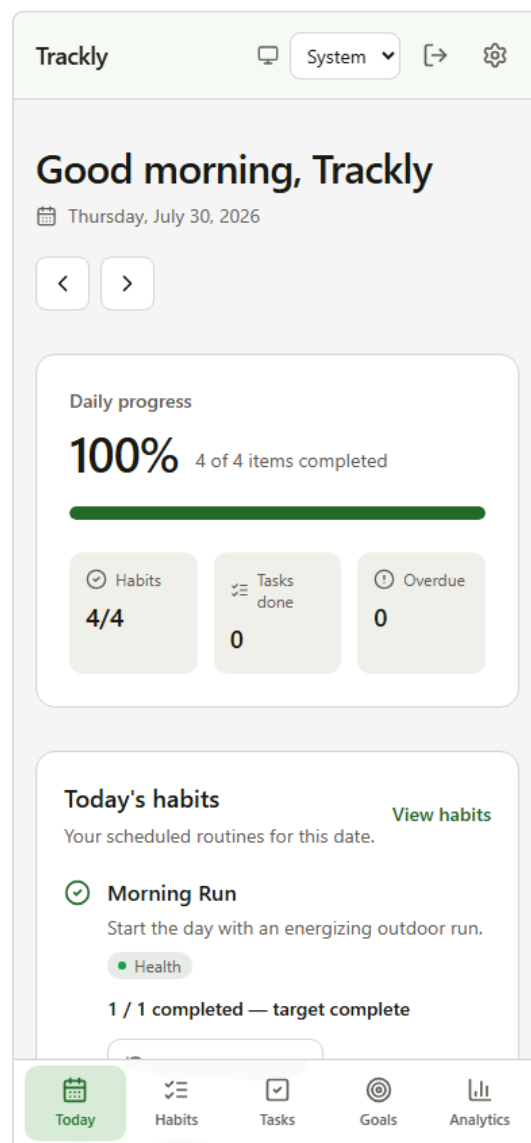


Figure 10.2 — Today Dashboard on Mobile

11 Habits

Habits support collection/detail reads, create, partial update, archive, restore, soft deletion, check-in, streak, category association, schedule, and reminder management. Collection controls are browser-native GET forms so view, date, search, sort, order, and page survive refresh and sharing.

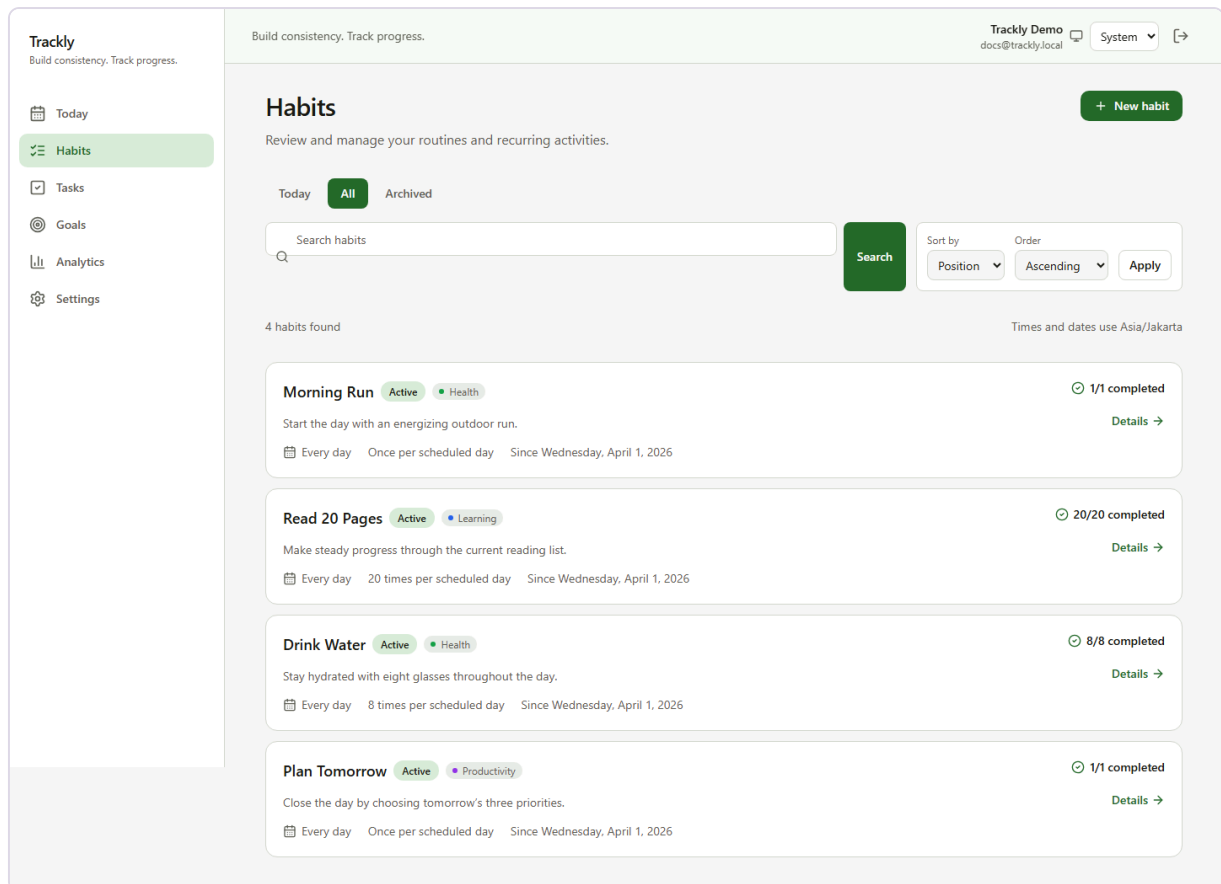


Figure 11.1 — Habit Collection and URL-Based Controls

Create and edit forms use Zod-backed React Hook Form validation. Daily, weekly, and custom schedules share backend schedule utilities.

Trackly
Build consistency. Track progress.

Build consistency. Track progress.

Trackly Demo
docs@trackly.local

System

Habits > New habit

New habit

Define a routine and choose when it should appear.

Habit details

Name *

Description

Category

No category

Daily target *

1

Number of completions required each scheduled day.

Schedule

Frequency *

Daily

Start date *

07/30/2026

End date

mm/dd/yyyy

Cancel Create habit

Figure 11.2 — Habit Creation Form

Check-ins send the final absolute `completedCount`, not an increment command. Target-one habits toggle between one and zero; multi-target habits increment or decrement within bounds. Zero removes the stored row.

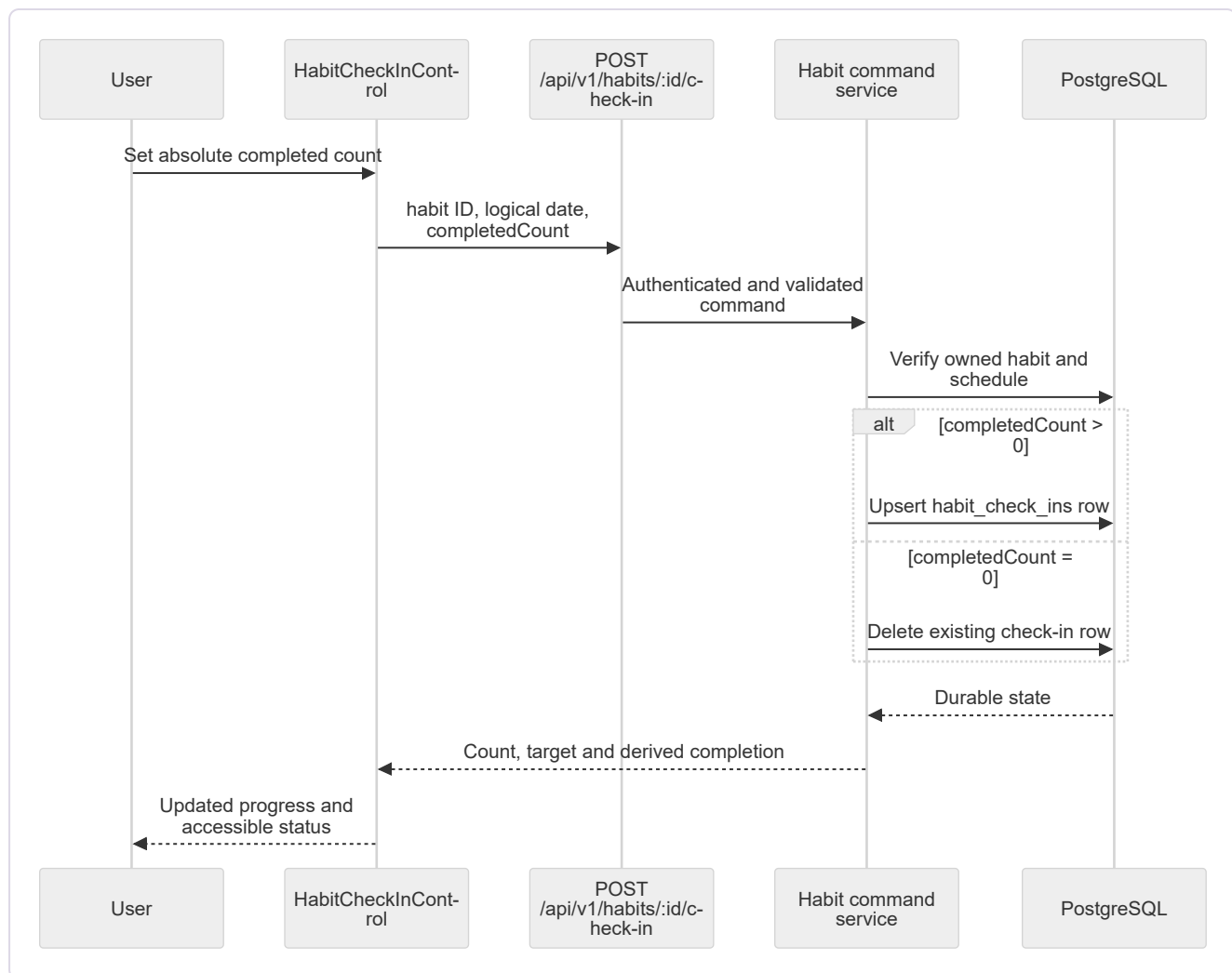


Figure 11.5 — Absolute Habit Check-In Flow

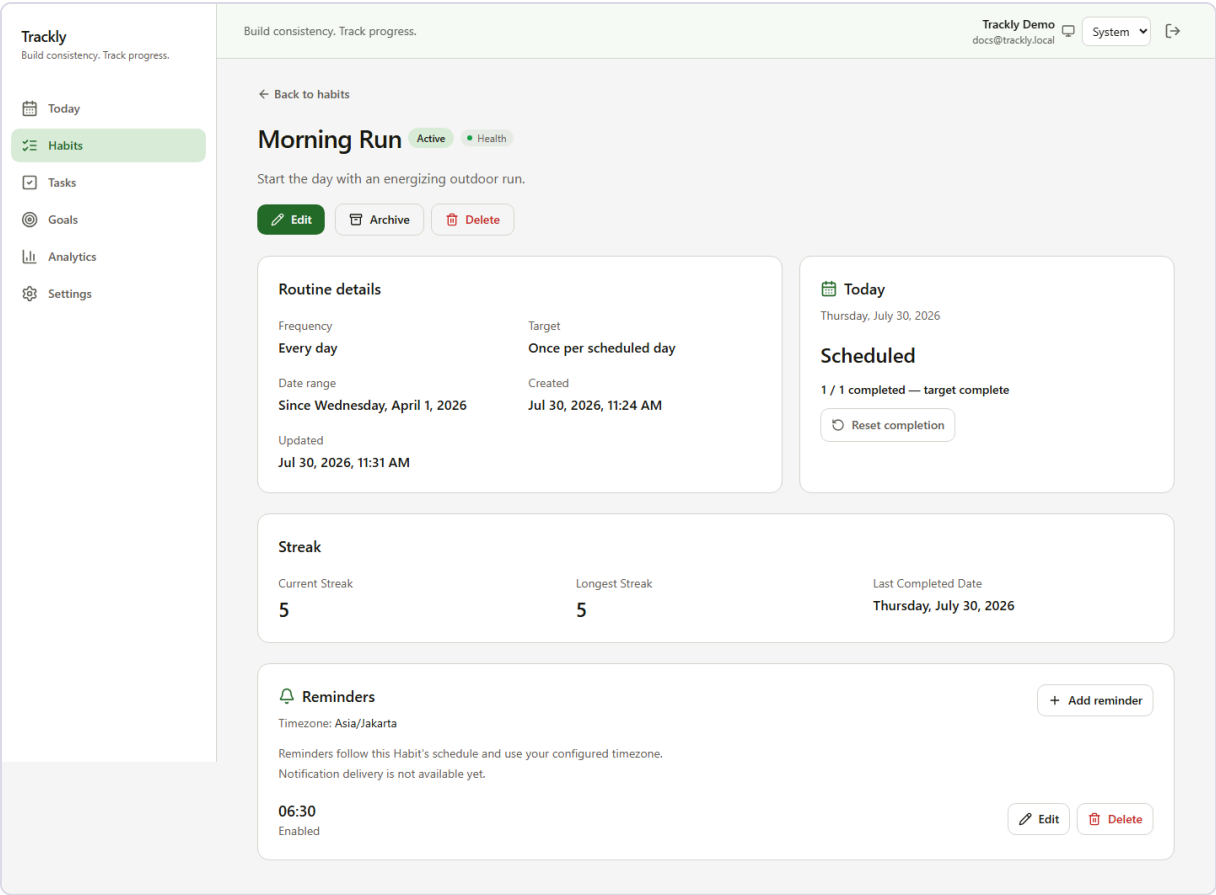


Figure 11.3 — Habit Detail, Check-In, and Streak

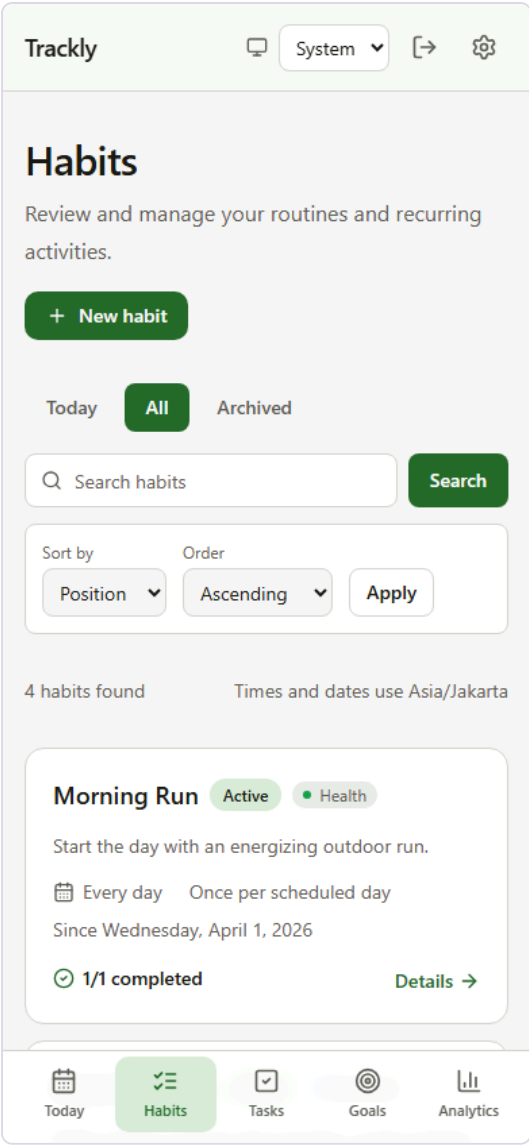


Figure 11.4 — Habit Collection on Mobile

12 Goals

Goals define finite accumulation targets linked to owned habits. Progress is calculated from habit check-ins inside the goal date range and capped only where the consuming metric requires it. The dashboard groups active goals by progress, deadline, target achievement, and over-target state.

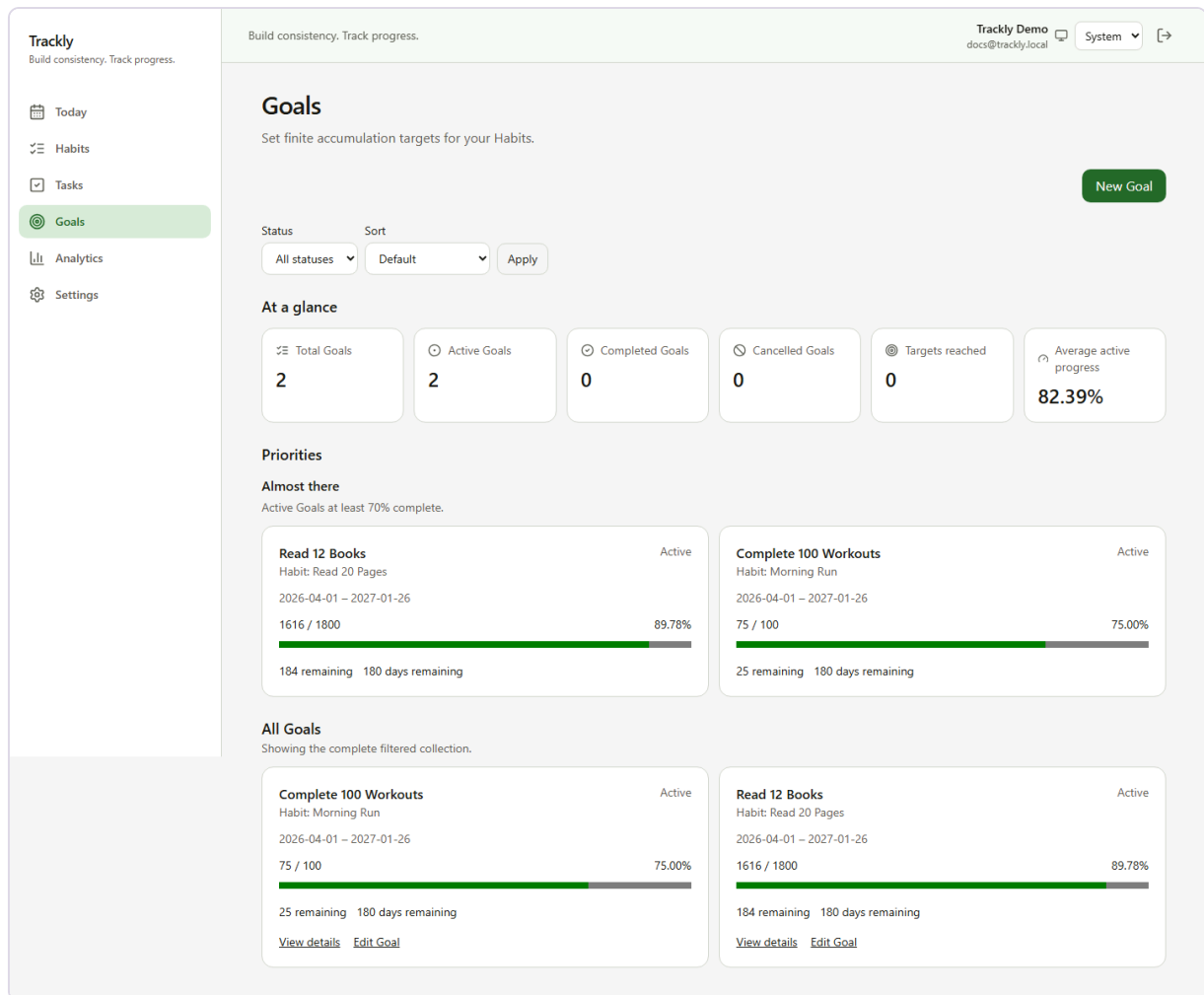


Figure 12.1 — Goal Dashboard and Derived Progress

Create, update, and delete operations verify that the linked habit and optional category belong to the authenticated user. Goal steps provide a separate checklist projection used by Today.

13 Categories and Preferences

Categories provide optional user-owned labels for habits and goals. The current public API lists active categories; it does not expose category-management commands.

Preferences store timezone, week start, date format, time format, and theme. Timezone is a domain input, while other preferences primarily affect presentation. Missing preference rows resolve documented defaults, and invalid legacy timezones safely fall back to UTC in dependent services.

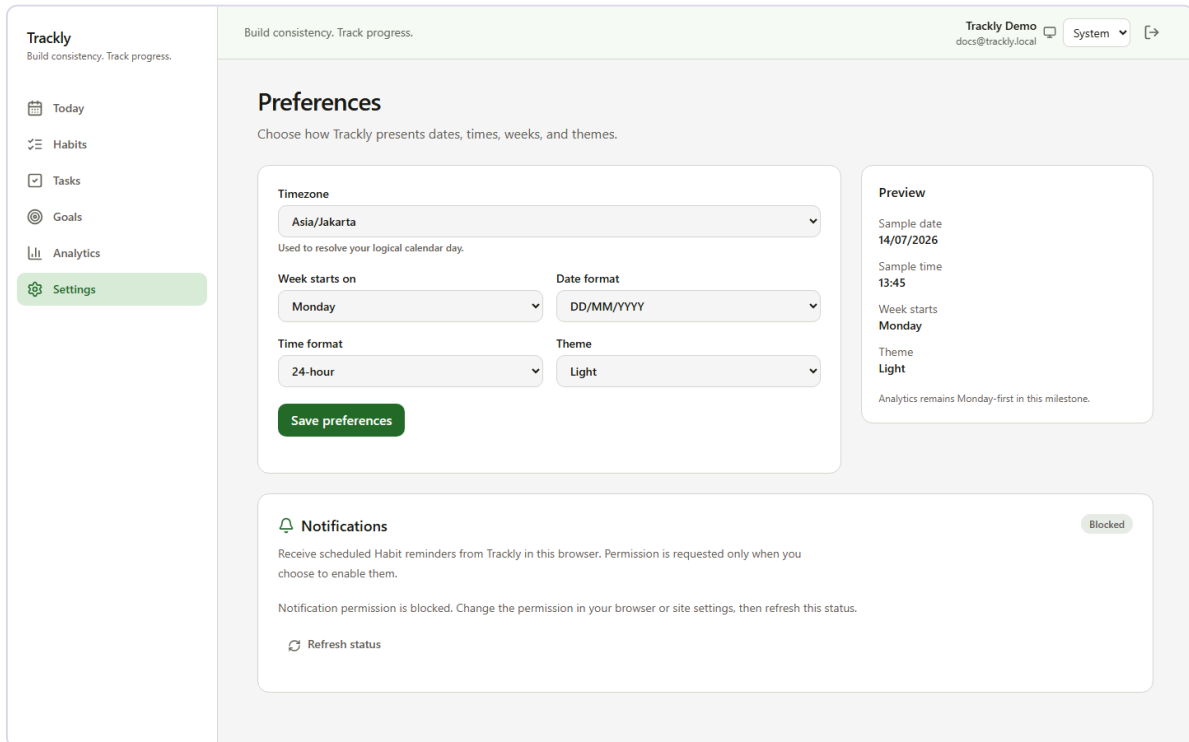


Figure 13.1 — Preferences and Device Notification Settings

14 Analytics

Analytics is read-only and calculated on demand. It reuses a bounded repository snapshot to serve summary, history, insights, heatmap, and rankings without six overlapping database reads.

Completion rate answers whether scheduled occurrences reached their target. Progress rate answers how much capped target volume was completed. These values intentionally differ for partial multi-target habits.

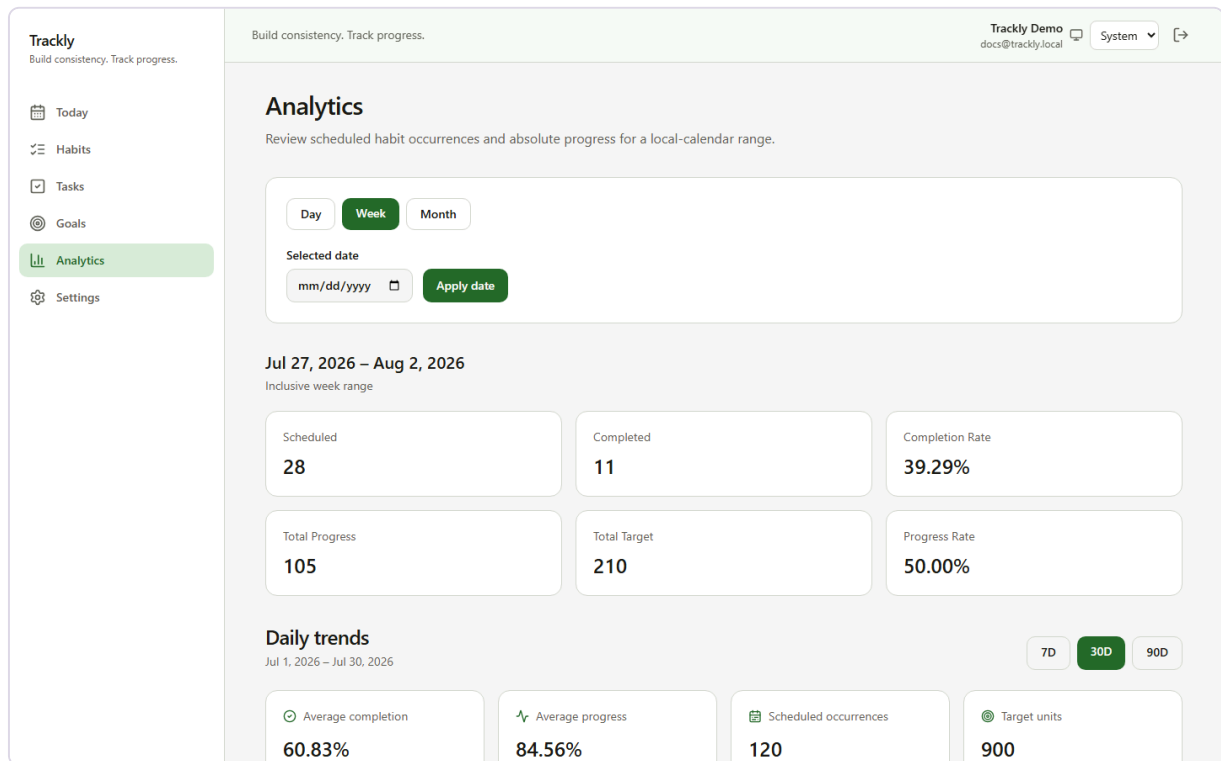


Figure 14.1 — Analytics Summary and Daily Trends

History includes every local date in its range, including zero-activity dates. Insights exclude inactive days where required, use deterministic tie-breaking, and compare equal recent/previous windows. The heatmap preserves the calendar shape while rankings remain user-scoped.

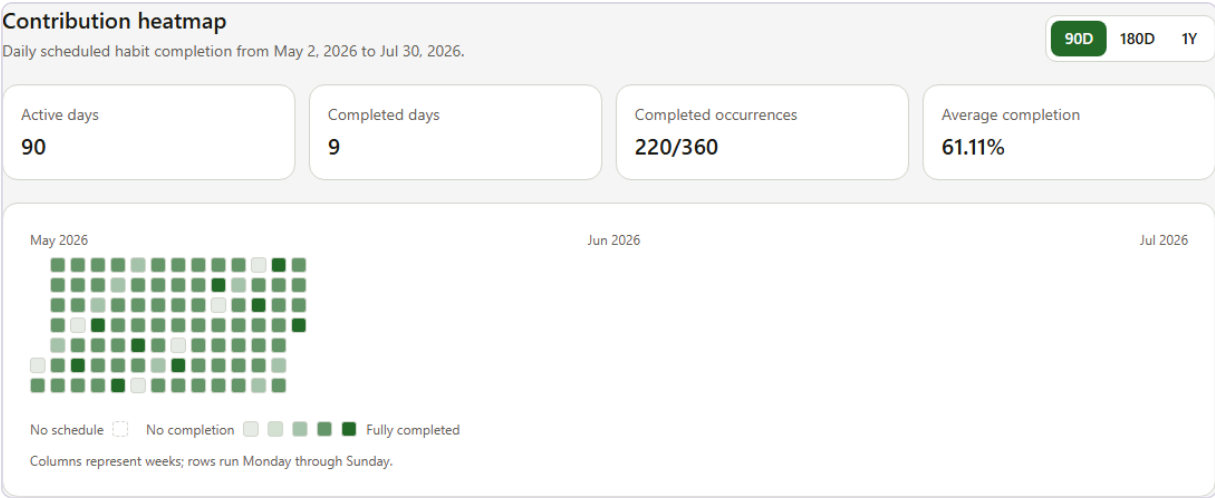


Figure 14.2 — Contribution Heatmap

15 Reminders and Web Push

Reminders attach local times to habits. Eligibility considers authenticated ownership, habit lifecycle, schedule, completion, date, timezone, and the configured reminder time. A separate scheduler evaluates due occurrences.

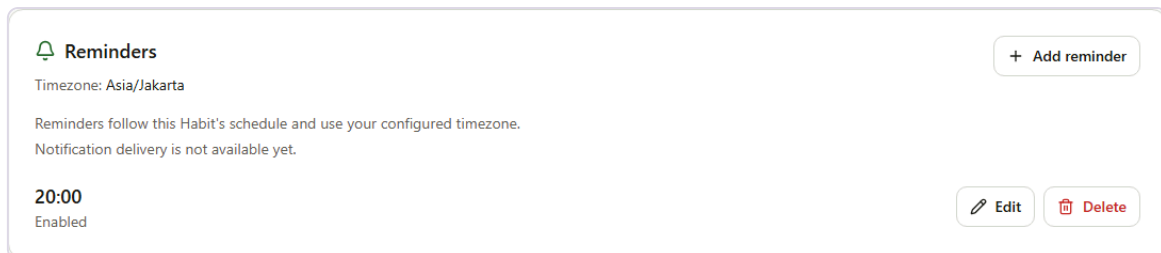


Figure 15.1 — Reminder Management on Habit Detail

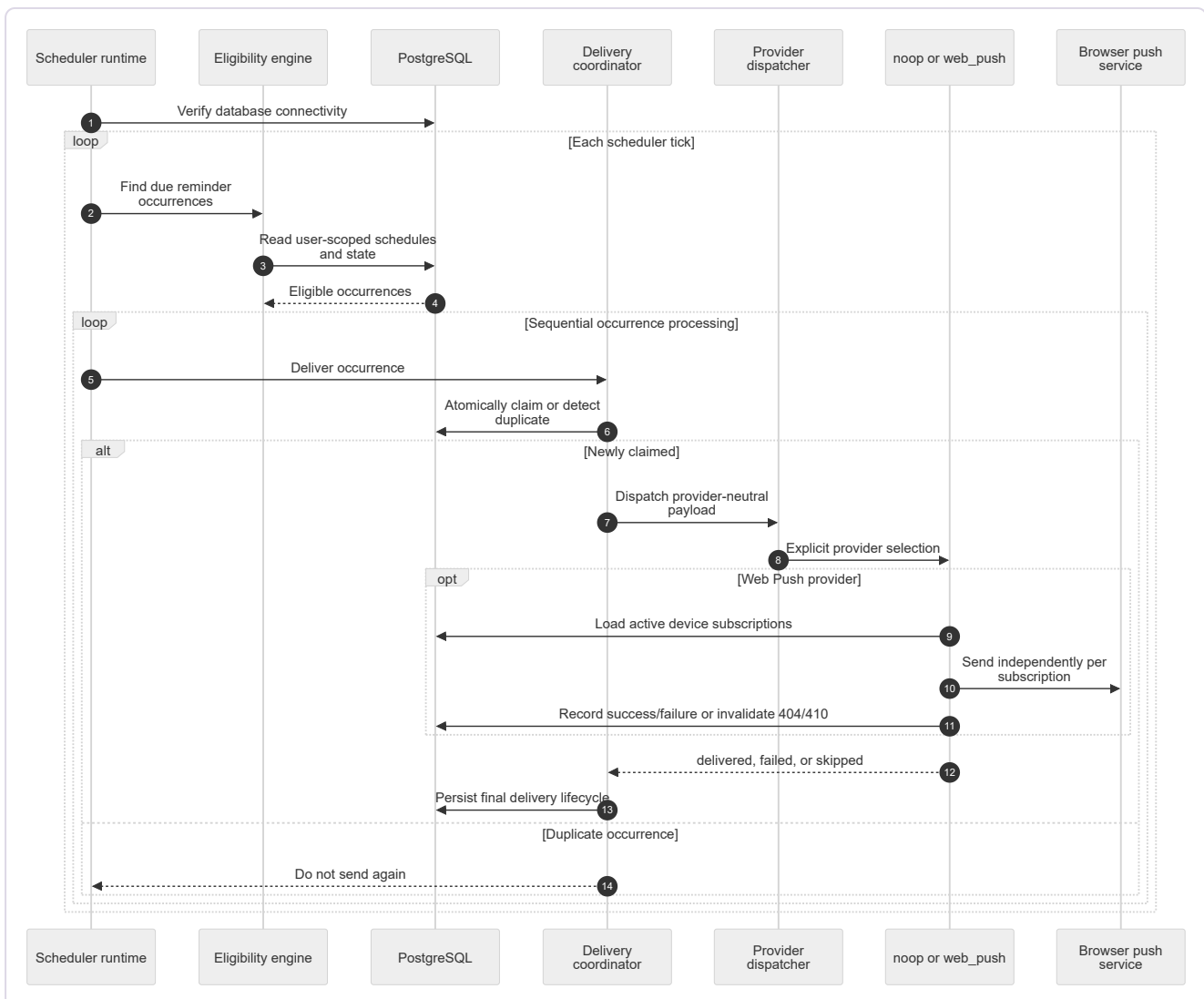


Figure 15.2 — Reminder Scheduling and Notification Delivery

Browser subscriptions are device-specific. Permission is requested only after an explicit action. The service worker displays minimal payloads and constructs only known internal navigation targets. HTTP 404/410 invalidates expired subscriptions; transient failures do not trigger automatic retry.

Part IV — Engineering

16 Local Development

Trackly uses pnpm workspaces. Node 24 matches the Docker build images, and pnpm 10.13.1 is declared by the root package.

```
corepack enable
pnpm install
Copy-Item .env.example .env
docker compose up --build
```

The frontend defaults to port 3000, backend to 4000, and PostgreSQL to 5432. Host processes use `localhost` ; containers use Compose service names.

17 Environment Configuration

Backend configuration is parsed and validated before startup. Production requires HTTPS authentication/trusted origins, a non-placeholder Better Auth secret, disabled diagnostics, and complete VAPID configuration when Web Push is used.

Frontend public configuration is limited to browser-safe URLs and the VAPID public key. `NEXT_PUBLIC_*` variables are visible to clients and must never contain database credentials, private VAPID material, or authentication secrets.

Security

Populate environment values through the deployment secret mechanism. Never commit a populated `.env` file.

18 Testing

Vitest covers backend services/routes, frontend components/services, and PostgreSQL integration behavior. Fastify injection avoids opening network ports for HTTP contract tests. External Web Push and browser APIs are mocked.

```
pnpm format:check
pnpm lint
pnpm typecheck
pnpm test:backend
pnpm test:frontend
pnpm test:database
pnpm build
docker compose config --quiet
git diff --check
```

The database suite remains separate from `pnpm validate` because it requires PostgreSQL. Publication screenshots use a dedicated Playwright workflow and reserved documentation account.

19 Debugging

Begin with the smallest boundary:

1. Capture the failing command, route, response status, and request ID.
2. Compare `/health` with `/ready`.
3. Inspect the relevant frontend, backend, PostgreSQL, or scheduler logs.
4. Verify host-versus-container URLs.
5. Reproduce with a focused test.

Public errors remain generic. Server logs may contain stack traces for unexpected failures, but redaction prevents credential and token fields from being logged.

20 Docker and Database Migrations

Development Compose starts PostgreSQL, waits for its health check, starts the backend, waits for backend readiness, and then starts the frontend.

```
pnpm db:generate  
pnpm db:migrate  
pnpm db:seed  
pnpm test:database
```

Schema changes require reviewed migrations. The production backend image is not a self-contained migration image because it does not copy the SQL migration directory; a deployment must supply a release workspace or purpose-built migration job.

Part V — Deployment and Operations

21 Production Deployment

The repository supplies multi-stage frontend and backend production Docker targets. Both run as non-root users. The frontend uses Next.js standalone output; the backend runs compiled JavaScript. PostgreSQL and the scheduler are separate runtime responsibilities.

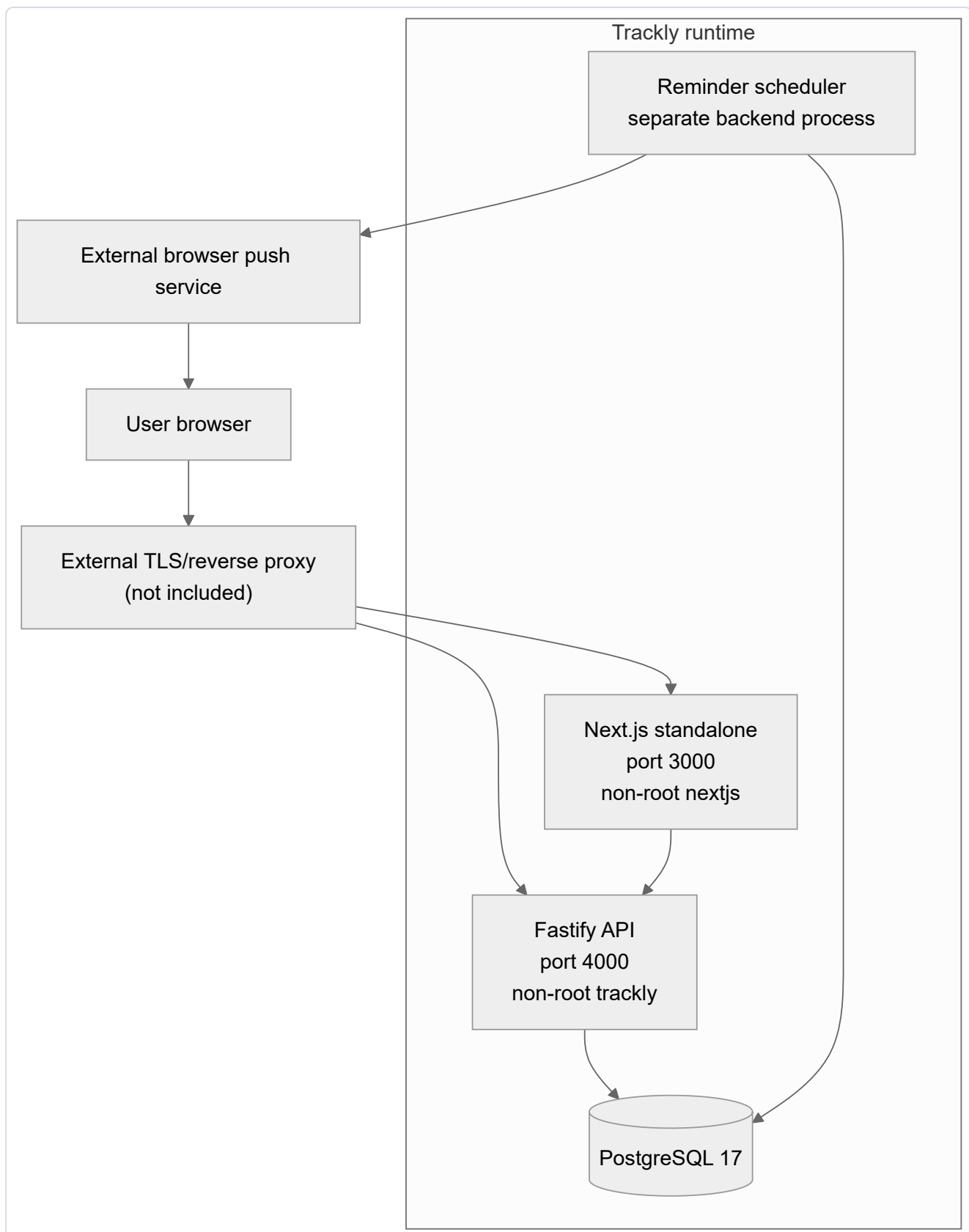
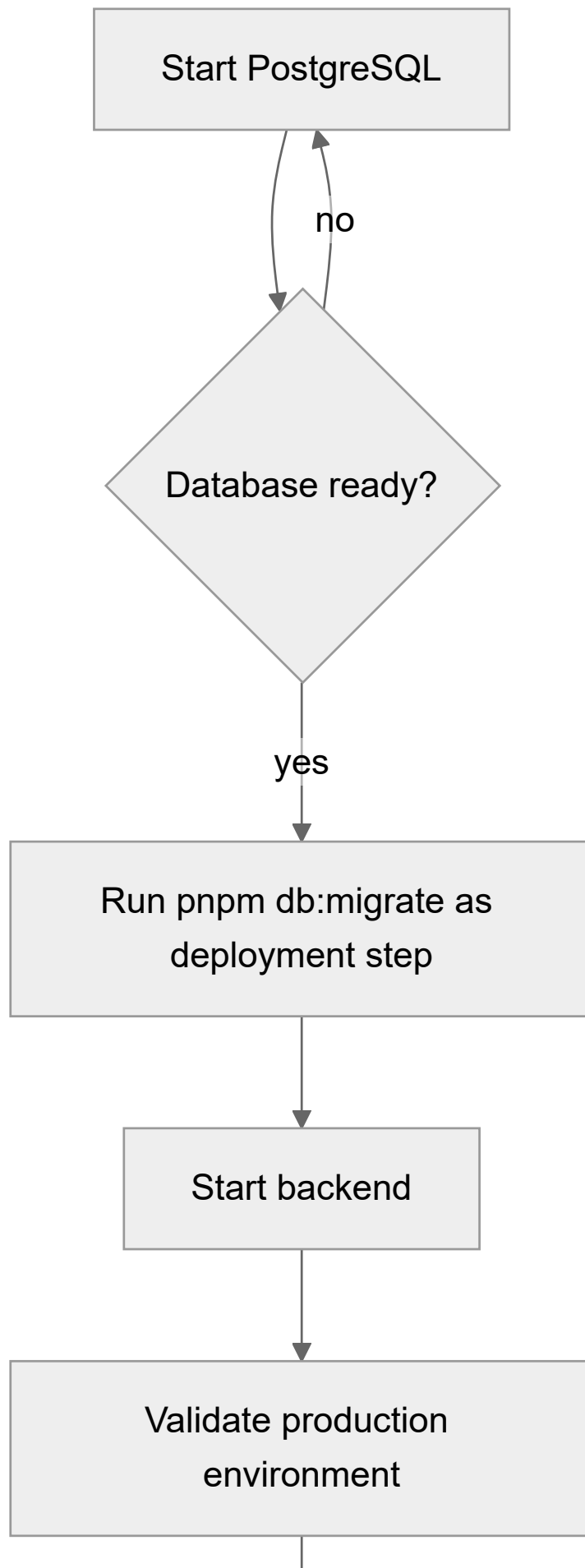


Figure 21.1 — Supported Deployment Topology and External Boundaries



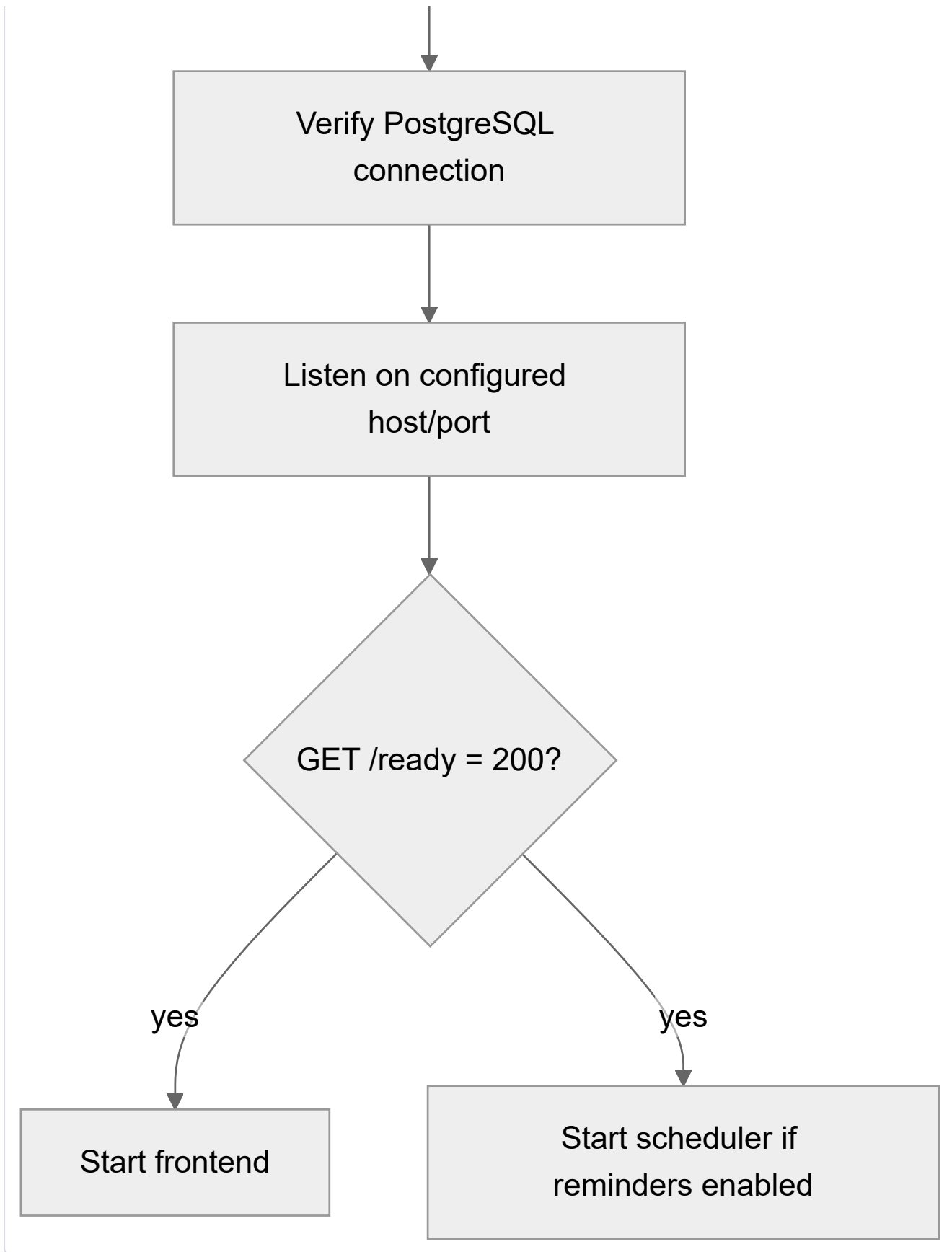


Figure 21.2 — Container Startup Dependencies

22 Reverse Proxy and Networking

No reverse proxy configuration is checked in. Production must supply TLS termination, preserve the host/protocol needed by Better Auth, route frontend and backend paths correctly, forward validated request IDs, and align CORS with trusted origins.

Static Next.js assets may use immutable caching where filenames are content addressed. Authenticated HTML and API responses must not enter shared caches. `TRUST_PROXY` should be enabled only behind a known proxy topology.

23 Monitoring and Logging

Pino emits readable development logs and machine-readable production JSON. Request completion records include timestamp, level, request ID, method, path, status, duration, and authenticated user ID when available. Audit events record actor, action, resource, outcome, and request ID.

Trackly provides `/health` for liveness and `/ready` for PostgreSQL readiness. It does not ship an external log platform, metrics, tracing, dashboards, or alerts. Operators must retain logs and alert on readiness failures, error rates, scheduler failures, and notification delivery outcomes.

24 Backup and Recovery

The development `postgres_data` volume provides persistence, not backup. The repository does not implement scheduled backups, retention, encryption, point-in-time recovery, or restore automation.

Production requires an external PostgreSQL backup plan with tested restore procedures. Recovery must restore schema and data coherently, apply only reviewed forward migrations, and verify authentication, ownership, check-ins, goals, reminders, and notification records.

25 Security and Operational Readiness

Implemented controls include Helmet security headers and CSP, CORS allowlists, HttpOnly Better Auth cookies, structured redaction, Zod validation, environment hardening, request IDs, process-local rate limiting, authenticated ownership, safe internal notification routing, and non-root production images.

Operational Limitations

Trackly does not include TLS/reverse proxy configuration, production orchestration, distributed rate limits, scheduler leadership, automated backups, external monitoring, CI/CD, or rollback automation.

A production review must define secret rotation, proxy trust, cookie behavior, database recovery objectives, migration ownership, one scheduler leader, central log retention, alerts, and capacity budgets.

Part VI — Reference

26 API Reference

The implemented API groups are:

Group	Namespace	Authentication
Health/readiness	<code>/health</code> , <code>/ready</code>	Public
Swagger	<code>/docs</code> when enabled	Public when exposed
Better Auth	<code>/api/auth/*</code>	Framework-specific
Current user	<code>/api/v1/auth/me</code>	Required
Today	<code>/api/v1/today</code>	Required
Categories	<code>/api/v1/categories</code>	Required
Habits and streak/check-in	<code>/api/v1/habits/*</code>	Required
Goals	<code>/api/v1/goals/*</code>	Required
Preferences	<code>/api/v1/preferences</code>	Required
Reminders	<code>/api/v1/habits/:habitId/reminders/*</code>	Required
Push subscriptions	<code>/api/v1/push-subscriptions</code>	Required
Analytics	<code>/api/v1/analytics/*</code>	Required

List endpoints document their own pagination, filtering, and deterministic sorting. Rate-limited routes return the standard 429 envelope. Swagger is generated from registered Fastify schemas and may be disabled in production.

For every request field, response model, error, and service mapping, use the [canonical API reference](#).

27 Database Reference

The schema contains fourteen implemented tables:

`user`, `session`, `account`, `verification`, `categories`, `habits`, `habit_schedules`, `habit_check_ins`, `tasks`, `goals`, `goal_steps`, `user_preferences`, `reminders`, `notification_deliveries`, and `push_subscriptions`.

Better Auth owns the first four authentication tables. Domain tables use database constraints, ownership indexes, and explicit cascade/set-null behavior. The task table exists, but the product workflow remains incomplete.

The [canonical database reference](#) lists every column, type, default, foreign key, index, unique constraint, check constraint, enumeration, relation, and lifecycle rule.

28 Troubleshooting

Symptom	First checks
Stack does not start	<code>docker compose config</code> , service status, port collisions, logs
Healthy but not ready	Database URL, PostgreSQL health, network, credentials, migrations
Authentication loop	Auth URLs, trusted origins, CORS, cookies, HTTPS
Frontend cannot call API	Browser URL versus internal server URL
HTTP 429	Configured window, duplicate submissions, trusted proxy
Scheduler does not deliver	Separate process, database, eligibility, provider, singleton ownership
Web Push unavailable	HTTPS/localhost, public/private VAPID config, browser permission
Build/test failure	Run formatting, lint, type, package tests, and build independently

Do not expose secrets in incident reports. Preserve timestamps, deployment version, route, and sanitized request IDs. Full procedures are in [Troubleshooting](#).

29 Frequently Asked Questions

29.1 Is analytics persisted?

No. Analytics, streaks, percentages, and goal progress are derived.

29.2 Does the backend start the scheduler?

No. The scheduler is a separate runtime.

29.3 Can a client choose a user ID?

No. Ownership always comes from the authenticated session.

29.4 Can `db:push` deploy production schema?

No. Production uses reviewed migrations.

29.5 Is a Compose volume a backup?

No. It is local persistence.

29.6 Does Trackly use Firebase?

No. Browser notifications use standard Web Push with VAPID.

See the complete [FAQ](#).

30 Glossary

Term	Meaning
Logical date	A <code>YYYY-MM-DD</code> calendar date interpreted in the user's timezone
Occurrence	A derived scheduled habit instance for one logical date
Check-in	Absolute progress stored for one habit/date
Active day	A date with at least one scheduled occurrence
Completion rate	Completed occurrences divided by scheduled occurrences
Progress rate	Capped completed volume divided by target volume
Soft delete	Retention through nullable <code>deleted_at</code>
Request ID	Correlation identifier returned in <code>x-request-id</code> and logs
Provider	Explicit notification delivery adapter such as <code>noop</code> or <code>web_push</code>
Readiness	Ability to query PostgreSQL and serve database-dependent traffic

See the complete [Glossary](#).

31 Decision Log

Current repository-backed decisions include:

1. TypeScript across application code.
2. Versioned Trackly application endpoints.
3. Controller, service, and repository separation.
4. PostgreSQL and Drizzle with reviewed migrations.
5. Better Auth ownership of authentication data.
6. User-timezone logical dates.
7. Derived metrics rather than persisted aggregates.
8. Server-rendered reads with focused client boundaries.
9. Provider-neutral notification delivery.
10. Structured logs and audit events without a persisted audit table.
11. Development Compose plus production image targets.

The [Decision Log](#) records context, consequences, and evidence without inventing dates or authors.

Appendices

Appendix A — Useful Commands

```
pnpm dev
pnpm validate
pnpm test:database
pnpm build
pnpm db:generate
pnpm db:migrate
pnpm db:seed
pnpm audit:security
pnpm docs:screenshots
docker compose up --build
docker compose config --quiet
git diff --check
```

Appendix B — Known Limitations

- Tasks and standalone Insights remain placeholder-only.
- Category management has no public mutation workflow.
- Analytics uses fixed periods rather than arbitrary date ranges.
- Web Push is the only real notification provider.
- Notification delivery has no automatic retry queue or history UI.
- Scheduler leadership is not distributed.
- Rate limiting is process-local.
- Swagger exposure, TLS, backups, monitoring, and production orchestration are deployment responsibilities.
- The production backend image is not a migration runner.
- PDF/DOCX and Mermaid-SVG generation are not yet automated.

Appendix C — Future Improvements

Repository-backed engineering needs include production deployment definition, migration packaging, backup/restore drills, external log retention and alerts, frontend health checks, scheduler ownership, distributed rate limiting for multi-replica deployment, performance budgets, and documentation publication automation.

No unimplemented product capability is presented as committed roadmap.

Appendix D — Documentation Maintenance

Canonical Markdown remains authoritative. When implementation changes:

1. Update the relevant canonical design, feature, development, deployment, API, or database document.
2. Update the publication chapter only when the change affects its narrative.
3. Regenerate deterministic screenshots when visible UI changes.
4. Update screenshot and figure manifests.
5. Validate relative paths, internal anchors, duplicate headings, fences, Mermaid rendering, CSS, and formatting.
6. Generate PDF and DOCX from reviewed Markdown.
7. Visually inspect every page before publication.

The [publication README](#) and [publication plan](#) define the next rendering stage.